Les bonnes pratiques de la programmation avec JS

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Introduction aux bonnes pratiques en JavaScript **4**

A. Les bonnes pratiques en JavaScript 4

B. Le guide de style Airbnb 5

C. Exercice : Quiz 6

III - Quelques bonnes pratiques en JavaScript **8**

A. Quelques bonnes pratiques en JavaScript 8

B. Exercice : Quiz 12

IV - L'essentiel **13**

V - Auto-évaluation **14**

A. Exercice 14

B. Test 15

Solutions des exercices **17**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 1 heure

Environnement de travail : VS Code

Contexte

L'écriture de code de qualité est essentielle pour garantir la fiabilité et les performances de vos applications en JavaScript. Les bonnes pratiques en JavaScript regroupent des règles et des conventions de codage qui peuvent vous aider à maintenir cette qualité élevée, à optimiser la performance et la sécurité, et à améliorer la maintenabilité de votre code.

Dans ce cours, nous allons passer en revue différentes bonnes pratiques recommandées pour le développement en JavaScript, ainsi que les avantages et les inconvénients de leur utilisation. Nous allons également explorer l'utilisation du guide de style Airbnb pour JavaScript, qui fournit des directives précises pour écrire du code clair, cohérent et facile à lire.

Ce cours vous permettra de dégrossir l'essentiel des règles de bonnes pratiques, mais gardez à l'esprit que ces bonnes pratiques peuvent évoluer, cela peut donc faire l'objet d'une veille au cours de votre vie professionnelle.

Tous droits réservés à STUDI - Reproduction interdite

II Introduction aux bonnes pratiques en JavaScript

A. Les bonnes pratiques en JavaScript

Az Définition

Les bonnes pratiques en JavaScript sont des conventions de codage qui permettent de maintenir la qualité du code, d'améliorer la lisibilité et la maintenabilité, et de réduire les erreurs et les bogues. Les avantages de suivre les bonnes pratiques sont nombreux, tels que :

- **Amélioration de la qualité du code** : les bonnes pratiques aident à écrire un code cohérent, organisé et facile à comprendre. Cela permet aux développeurs de travailler plus efficacement et de réduire le temps nécessaire pour déboguer le code.
- **Réduction des erreurs et des bogues** : les bonnes pratiques en JavaScript permettent de détecter les erreurs de code et les bogues plus rapidement, ce qui facilite leur correction. Cela permet également d'améliorer la fiabilité de l'application et d'augmenter la satisfaction des utilisateurs.
- **Facilitation de la collaboration** : les bonnes pratiques en JavaScript permettent aux membres de l'équipe de travailler plus efficacement et de collaborer plus facilement. En suivant les mêmes conventions de codage, les membres de l'équipe peuvent comprendre plus facilement le code écrit par les autres.

Cependant, il y a également quelques inconvénients à suivre les bonnes pratiques en JavaScript :

- **Temps supplémentaire nécessaire lors du développement** : le suivi des bonnes pratiques en JavaScript peut nécessiter un temps supplémentaire pour mettre en place les outils nécessaires et pour apprendre à les utiliser correctement. Cependant, ce temps supplémentaire est souvent compensé par une réduction des erreurs et des bogues.
- **Possibilité de surcharge cognitive** : si les bonnes pratiques sont trop complexes ou difficiles à comprendre, cela peut entraîner une surcharge cognitive pour les développeurs. Cela peut rendre le développement plus difficile et moins efficace. Il est donc important de mettre en place des bonnes pratiques qui correspondent à l'équipe, et qu'elles ne deviennent pas un fardeau pour les développeurs.

Refactoring

Az Définition

Le *refactoring* (ou « *remaniement de code* » en français) est le processus consistant à modifier le code source d'un programme sans en changer le comportement externe, dans le but d'améliorer sa lisibilité, sa maintenabilité, sa qualité et/ou ses performances. Le refactoring ne consiste pas à ajouter de nouvelles fonctionnalités ou à corriger des bogues, mais plutôt à optimiser la structure du code existant.

Le refactoring peut inclure des tâches telles que la suppression de code inutile, le renommage de variables, la réorganisation de fonctions et de classes pour faciliter leur compréhension, la normalisation de la syntaxe, la simplification des expressions, l'extraction de méthodes et la réduction des dépendances entre les différentes parties du code.

Tous droits réservés à STUDI - Reproduction interdite

B. Le guide de style Airbnb

Le guide de style Airbnb est un ensemble de règles et de recommandations pour écrire du code JavaScript clair, cohérent et facile à lire. Il est utilisé par de nombreuses entreprises et développeurs pour maintenir la qualité du code et améliorer la lisibilité et la maintenabilité. Il est, comme son nom l'indique, créé par Airbnb.

Il est publié sur GitHub sous la forme d'un référentiel open source (GitHub[1]) et est conçu pour être utilisé avec un outil appelé ESLint, qui est un linter JavaScript qui peut identifier et signaler certaines erreurs dans votre code. Il est accessible aussi *via* ce lien : Airbnb JavaScript Style Guide[2].

En utilisant le guide de style Airbnb avec ESLint, les développeurs peuvent vérifier leur code en temps réel pour s'assurer qu'il respecte les normes de codage et les bonnes pratiques recommandées par Airbnb. Cela permet de maintenir la qualité du code, d'améliorer la lisibilité et la maintenabilité, et de réduire les erreurs et les bogues.

Le JavaScript style guide Airbnb couvre différents aspects : la déclaration de variables, la gestion des erreurs, la structure des fichiers, les fonctions, les classes, les chaînes de caractères, les tableaux, les objets, et bien plus encore. Le guide de style Airbnb fournit également des exemples de code pour illustrer les conventions de codage recommandées.

Le JavaScript style guide Airbnb est un outil très utile pour les développeurs JavaScript, car il fournit des normes de codage cohérentes et recommandées pour écrire du code clair, cohérent et facile à lire. Cela évite aux équipes de créer leurs propres conventions, qui seraient donc différentes d'une entreprise à l'autre.

Ainsi, les bonnes pratiques en JavaScript sont essentielles pour maintenir la qualité du code, améliorer la lisibilité et la maintenabilité, et réduire les erreurs et les bogues. Le guide de style Airbnb est un outil utile pour maintenir la qualité du code en fournissant des règles et des conventions de codage claires et cohérentes. Bien que l'utilisation du guide de style Airbnb puisse avoir quelques inconvénients, les avantages l'emportent largement sur les inconvénients, en particulier pour les équipes de développement travaillant sur de grands projets.

ESLint

⊕ Complément

ESLint est un outil d'analyse statique de code open source qui permet de détecter et de signaler les problèmes de code JavaScript, tels que les erreurs de syntaxe, les erreurs de style et les problèmes liés à la qualité du code. Le but principal de cet outil est d'aider les développeurs à écrire du code JavaScript propre, cohérent et sans erreurs.

ESLint utilise un système de règles configurables pour vérifier votre code et vous fournir des rapports sur les problèmes rencontrés. Vous pouvez configurer ces règles en fonction de vos préférences ou des normes de votre équipe de développement. En outre, vous pouvez étendre les fonctionnalités d'ESLint en utilisant des plugins ou en écrivant vos propres règles personnalisées.

L'intégration d'ESLint dans votre flux de travail de développement peut vous aider à maintenir la qualité du code, à détecter les erreurs potentielles avant l'exécution et à faciliter la collaboration entre les membres de l'équipe en favorisant un style de code cohérent.

1. https://github.com/airbnb/javascript
2. https://airbnb.io/javascript/

Tous droits réservés à STUDI - Reproduction interdite

Comment utiliser ESLint

Méthode

Installez ESLint : pour installer ESLint, vous devez utiliser un gestionnaire de paquets tel que npm ou yarn (ESLint [3]). Si vous utilisez npm, vous pouvez installer ESLint en exécutant la commande suivante dans votre terminal :

```
npm init @eslint/config
```

Exécutez ESLint : une fois que vous avez configuré ESLint, vous pouvez exécuter ESLint en utilisant la commande suivante dans votre terminal :

```
npx eslint yourfile.js
```

Où « *yourfile.js* » est le fichier que vous souhaitez analyser.

Vous pouvez également exécuter ESLint sur plusieurs fichiers en même temps en spécifiant des patterns de fichiers :

```
npx eslint "src/**/*.js"
```

Cette commande exécutera ESLint sur tous les fichiers .js dans le dossier src et ses sous-dossiers.

Un retour de ESLint

Exemple

Si vous avez un fichier JS avec :

```
function test() {
  var a = 1;
  var _b = 2;
  console.log(a)
}

test();
```

Lorsque vous exécutez ESLint sur ce fichier avec la configuration par défaut, vous obtenez le retour suivant :

```
  2:7   error  'a' is assigned a value but never used       no-unused-vars
  3:7   warning  '_b' is defined but never used             no-unused-vars
```

Cela indique qu'il y a 2 erreurs, l'une pour la variable a qui est toujours inutilisée, et une alerte pour la variable _b qui est définie, mais jamais utilisée, en fonction de la configuration des règles.

Vous allez pouvoir ensuite configurer votre ESLint, pour vérifier ou non certaines erreurs. Par exemple, en ajoutant la règle `'no-var'`, ESLint avertira chaque fois que vous déclarez une variable avec le mot clé `var`, ce qui est une mauvaise pratique. Avec le même code JavaScript, nous aurions donc ce retour :

```
2:3 Unexpected var, use let or const instead.  (no-var)
3:3 Unexpected var, use let or const instead.  (no-var)
3:7 '_b' is assigned a value but never used.  (no-unused-vars)
```

C. Exercice : Quiz

[solution n°1 p. 17]

Question 1

Le refactoring est le processus consistant à améliorer la qualité du code sans changer son comportement externe.

3. https://eslint.org/docs/latest/use/getting-started

Tous droits réservés à STUDI - Reproduction interdite

Vrai

Faux

Question 2

Quel est l'avantage principal de l'utilisation de bonnes pratiques en JavaScript ?

Améliorer la performance du code

Faciliter la maintenance du code

Rendre le code plus court

Question 3

ESLint est un outil qui permet de détecter les erreurs de syntaxe et les erreurs potentielles dans le code JavaScript.

Vrai

Faux

Question 4

Quel est le but principal du guide Airbnb de JavaScript ?

Fournir des directives pour écrire du code JavaScript conforme aux normes de codage de l'entreprise Airbnb

Fournir des directives pour écrire du code JavaScript conforme aux normes de codage de la communauté JavaScript

Fournir des directives pour écrire du code JavaScript conforme aux normes de codage de la W3C

Question 5

Le guide Airbnb de JavaScript diminue les performances en travail de groupe, mais augmente les performances en travail individuel.

Vrai

Faux

Tous droits réservés à STUDI - Reproduction interdite

III Quelques bonnes pratiques en JavaScript

A. Quelques bonnes pratiques en JavaScript

Utiliser const et let au lieu de var

Méthode

Il est recommandé d'utiliser `const` et `let` pour déclarer les variables en JavaScript, plutôt que `var`. Les variables déclarées avec `const` ne peuvent pas être réaffectées et les variables déclarées avec `let` peuvent être réaffectées. En revanche, les variables déclarées avec `var` ont une portée de fonction et peuvent être réaffectées. Voici un exemple :

```
// Bonne pratique
const PI = 3.14159;
let message = "Bonjour";

// Mauvaise pratique
var x = 10;
```

Utiliser des fonctions fléchées

Méthode

Les fonctions fléchées sont une nouvelle syntaxe pour définir des fonctions en JavaScript. Elles ont l'avantage de rendre le code plus concis et plus facile à lire. Voici un exemple :

```
// Bonne pratique
const add = (a, b) => a + b;

// Mauvaise pratique
function add(a, b) {
return a + b;
}
```

Utiliser les templates strings

Méthode

Les templates strings sont une nouvelle syntaxe pour créer des chaînes de caractères en JavaScript. Elles permettent d'incorporer des expressions JavaScript dans une chaîne de caractères. Voici un exemple :

```
// Bonne pratique
const name = "John";
const message = `Bonjour, ${name}!`;

// Mauvaise pratique
const name = "John";
const message = "Bonjour, " + name + "!";
```


Tous droits réservés à STUDI - Reproduction interdite

Éviter les boucles for-in pour les tableaux

Méthode

Il est recommandé d'éviter les boucles `for-in` pour parcourir les tableaux en JavaScript, car elles peuvent engendrer des erreurs. Il est préférable d'utiliser des boucles `for-of` ou `forEach`. Voici un exemple :

```
// Bonne pratique
const fruits = ["pomme", "banane", "orange"];

for (const fruit of fruits) {
console.log(fruit);
}

fruits.forEach(fruit => console.log(fruit));
```

```
// Mauvaise pratique
const fruits2 = ["pomme", "banane", "orange"];

for (const index in fruits2) {
console.log(fruits2[index]);
}
```

Éviter les fonctions constructeurs

Méthode

Il est recommandé d'éviter les fonctions constructeurs pour créer des objets en JavaScript, car elles peuvent engendrer des erreurs. Il est préférable d'utiliser des classes ou des objets littéraux. Voici un exemple :

```
// Bonne pratique
class Person {
constructor(name) {
this.name = name;
}
}

const john = new Person("John");

const person = { name: "John" };
```

```
// Mauvaise pratique
function Person(name) {
this.name = name;
}

const john = new Person("John");

const person = new Object({ name: "John" });
```

Tous droits réservés à STUDI - Reproduction interdite

Utiliser les méthodes Array

Méthode

Les méthodes Array sont des méthodes intégrées qui permettent de manipuler les tableaux en JavaScript. Il est recommandé de les utiliser plutôt que de parcourir les tableaux avec des boucles. Voici des exemples de méthodes Array utiles :

```
// filter : permet de filtrer les éléments d'un tableau selon un critère
const numbers1 = [1, 2, 3, 4, 5];

const evenNumbers = numbers1.filter(number => number % 2 === 0)
console.log(evenNumbers); // [2, 4]

// map : permet de transformer chaque élément d'un tableau en un autre élément
const numbers2 = [1, 2, 3, 4, 5];

const doubledNumbers = numbers.map(number => number * 2);
console.log(doubledNumbers); // [2, 4, 6, 8, 10]

// reduce : permet de réduire les éléments d'un tableau en une seule valeur
const numbers3 = [1, 2, 3, 4, 5];

const sum = numbers.reduce((accumulator, number) => accumulator + number, 0);
console.log(sum); // 15
```

Éviter les boucles imbriquées

Méthode

Il est recommandé d'éviter les boucles imbriquées en JavaScript, car elles peuvent ralentir les performances de l'application. Il est préférable d'utiliser des méthodes Array comme `map`, `filter` ou `reduce` pour éviter les boucles imbriquées. Voici un exemple :

```
// Bonne pratique
const matrix = [[1, 2], [3, 4], [5, 6]];

const flattenedMatrix = matrix.flatMap(row => row);
console.log(flattenedMatrix); // [1, 2, 3, 4, 5, 6]
```

```
// Mauvaise pratique
const matrix = [[1, 2], [3, 4], [5, 6]];
const flattenedMatrix = [];

for (let i = 0; i < matrix.length; i++) {
for (let j = 0; j < matrix[i].length; j++) {
flattenedMatrix.push(matrix[i][j]);
}
}

console.log(flattenedMatrix); // [1, 2, 3, 4, 5, 6]
```


Tous droits réservés à STUDI - Reproduction interdite

Éviter les évaluations en cascade

Méthode

Il est recommandé d'éviter les évaluations en cascade en JavaScript, car elles peuvent rendre le code difficile à lire et à maintenir. Il est préférable d'utiliser des méthodes Array ou des fonctions pour simplifier le code. Voici un exemple :

```
// Bonne pratique
const person = {
name: "John",
age: 30,
address: {
city: "Paris",
country: "France"
}
};

const country = person.address?.country;

// Mauvaise pratique
const person = {
name: "John",
age: 30,
address: {
city: "Paris",
country: "France"
}
};

const country = person.address && person.address.country;
```

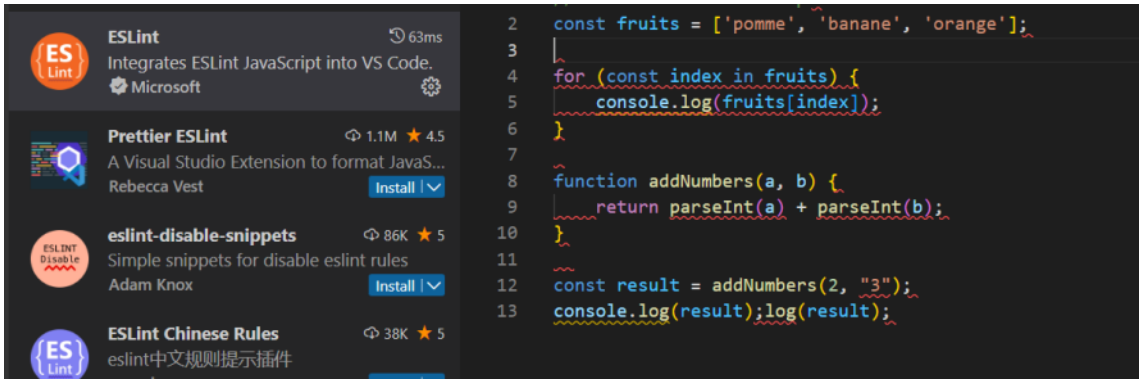
Complément

Voici une liste plus exhaustive des bonnes pratiques en JavaScript : developer.mozilla.org[4].

Les extensions

Complément

Beaucoup d'extensions existent pour nous signaler les erreurs directement depuis l'éditeur de code. Elles vous seront très pratiques, car elles vous éviteront de devoir taper les commandes ESlint en console. L'extension ESLint développée par Microsoft pourra par exemple vous aider sur VS Code, car elle soulignera le code présentant des erreurs en rouge.



4. https://developer.mozilla.org/fr/docs/MDN/Writing_guidelines/Writing_style_guide/Code_style_guide/JavaScript

Tous droits réservés à STUDI - Reproduction interdite

B. Exercice : Quiz

[solution n°2 p. 18]

Question 1

Il est recommandé d'utiliser des variables globales en JavaScript autant que possible.

- Vrai
- Faux

Question 2

Il est important d'utiliser des commentaires dans le code JavaScript pour expliquer le but et le fonctionnement du code.

- Vrai
- Faux

Question 3

Quelle sont les meilleures façons de déclarer une variable en JavaScript ?

- **A** `let variableName;`
- **B** `var variableName;`
- **C** `const variableName;`

Question 4

Les boucles imbriquées sont une bonne pratique.

- Vrai
- Faux

Question 5

Il est recommandé d'utiliser des boucles `for-in` pour itérer sur les tableaux en JavaScript.

- Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

IV L'essentiel

Nous avons appris l'importance des bonnes pratiques de développement et du respect des normes de codage pour garantir un code robuste, fiable et facilement maintenable. Nous avons vu comment utiliser des outils tels que le guide de style JavaScript d'Airbnb pour mettre en place des conseils de bonnes pratiques et aider les développeurs à écrire un code cohérent et facilement compréhensible.

Nous avons appris comment utiliser des outils de linting tels que ESLint pour détecter les erreurs de syntaxe, les problèmes de style et les mauvaises pratiques de codage dans notre code. Nous avons vu comment configurer ces outils pour qu'ils respectent les normes de codage de notre entreprise et comment les intégrer dans notre flux de travail de développement.

En résumé, la mise en place de bonnes pratiques de développement et le respect des normes de codage sont des éléments clés pour garantir un code robuste et facilement maintenable. Les outils tels que le guide de style JavaScript d'Airbnb ou ESLint peuvent nous aider à atteindre ces objectifs en fournissant des conseils de bonnes pratiques, en détectant les erreurs de codage.

Tous droits réservés à STUDI - Reproduction interdite

V Auto-évaluation

A. Exercice

Vous êtes développeur dans une entreprise, et vous recevez cet e-mail de votre leader technique.

Objet : refactoring fichier JS pour se conformer aux bonnes pratiques

Bonjour,

J'espère que vous allez bien. Nous avons récemment identifié un fichier JavaScript dans notre codebase qui ne suit pas les bonnes pratiques de codage. Nous aimerions que vous corrigiez ce fichier en suivant les directives ci-dessous.

Le fichier en question présente plusieurs problèmes :

- Il utilise `var` pour déclarer des variables au lieu de `let` ou `const`,
- Il y a des boucles imbriquées,
- Il utilise une boucle `for-in` pour parcourir un tableau.

Question 1

[solution n°3 p. 20]

Votre mission consiste à revoir le fichier et à apporter les modifications nécessaires pour le mettre en conformité avec nos standards de développement.

Voici le contenu du fichier HTML et du fichier JS concernés :

index.html :

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Exemple de page</title>
</head>
<body>
    <h1>Exemple de page HTML</h1>
    <ul id="result"></ul>

    <script src="script.js"></script>
</body>
</html>
```

script.js :

```javascript
var data = [
    {id: 1, value: "Item 1"},
    {id: 2, value: "Item 2"},
    {id: 3, value: "Item 3"}
];

var resultElement = document.getElementById("result");

function processItems() {
    for (var key in data) {
        var item = data[key];
        var listItem = document.createElement("li");
        listItem.innerHTML = "ID : " + item.id + ", Value : " + item.value;
        resultElement.appendChild(listItem);

        for (var i = 0; i < data.length; i++) {
            if (data[i].id === item.id) {
```


Tous droits réservés à STUDI - Reproduction interdite

```
                data[i].processed = true;
            }
        }
    }
}

function checkProcessedItems() {
    var allProcessed = true;
    for (var key in data) {
        allProcessed = allProcessed && data[key].processed;
    }
    return allProcessed ? "All items processed" : "Some items not processed";
}

processItems();
console.log(checkProcessedItems());
```

N'hésitez pas à me contacter si vous avez des questions ou besoin de clarifications. Une fois les modifications effectuées, merci de soumettre vos changements pour revue.

Cordialement

Vous êtes développeur dans une entreprise de vente de vêtements et de chaussures en ligne et votre responsable vous confie une mission.

Question 2

[solution n°4 p. 20]

Votre responsable vous annonce que l'un des stagiaires du département avait commencé à rédiger du code pour afficher une liste de produits et leur prix avant la fin de sa période de stage. Toutefois, après un rapide coup d'œil, vous vous apercevez que ce code ne respecte pas les bonnes pratiques de la programmation en JavaScript.

Votre mission va donc être de corriger le code suivant :

```
const products = [
  { name: 'T-shirt', price: 20 },
  { name: 'Jeans', price: 50 },
  { name: 'Sneakers', price: 80 }
];

for (const index in products) {
  console.log(products[index].name + ' - ' + products[index].price);
}
```

B. Test

Exercice 1 : Quiz

[solution n°5 p. 21]

Question 1

L'utilisation de bonnes pratiques en JavaScript peut réduire le temps de développement.

- Vrai
- Faux

Question 2

L'utilisation de bonnes pratiques en JavaScript peut améliorer la qualité du code.

Tous droits réservés à STUDI - Reproduction interdite

- Vrai
- Faux

Question 3

Quel est le but principal d'ESLint ?

- Traduire le code JavaScript
- Faire de l'autocomplétion dans le code JavaScript
- Fournir des suggestions pour améliorer la qualité du code JavaScript

Question 4

Quel est l'avantage de l'utilisation de `let` et `const` plutôt que `var` en JavaScript ?

- Elle permet de définir des variables globale
- Elle permet de définir des variables locales
- Elle permet de mieux contrôler la portée des variables

Question 5

Quel est l'avantage de l'utilisation de la méthode `map` pour itérer sur les tableaux en JavaScript ?

- Elle permet de modifier les éléments du tableau
- Elle permet de filtrer les éléments du tableau
- Elle permet de créer un nouveau tableau en appliquant une fonction à chaque élément du tableau d'origine

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 6]

Question 1

Le refactoring est le processus consistant à améliorer la qualité du code sans changer son comportement externe.

- ✓ Vrai
- Faux

Le refactoring permet d'améliorer la qualité du code en le rendant plus lisible, plus maintenable et plus performant, sans changer son comportement externe.

Question 2

Quel est l'avantage principal de l'utilisation de bonnes pratiques en JavaScript ?

- Améliorer la performance du code
- ✓ Faciliter la maintenance du code
- Rendre le code plus court

Les bonnes pratiques en JavaScript, telles que la modularisation du code, la gestion des erreurs et l'utilisation de noms de variables significatifs, permettent de rendre le code plus facile à comprendre et à modifier.

Question 3

ESLint est un outil qui permet de détecter les erreurs de syntaxe et les erreurs potentielles dans le code JavaScript.

- ✓ Vrai
- Faux

ESLint est un outil de linting qui permet de détecter les erreurs de syntaxe et les erreurs potentielles dans le code JavaScript, en se basant sur un ensemble de règles définies par l'utilisateur.

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Quel est le but principal du guide Airbnb de JavaScript ?

- ✓ Fournir des directives pour écrire du code JavaScript conforme aux normes de codage de l'entreprise Airbnb
- Fournir des directives pour écrire du code JavaScript conforme aux normes de codage de la communauté JavaScript
- Fournir des directives pour écrire du code JavaScript conforme aux normes de codage de la W3C

Le but principal du guide Airbnb de JavaScript est de fournir des directives pour écrire du code JavaScript conforme aux normes de codage de l'entreprise Airbnb. Le guide Airbnb est un ensemble de bonnes pratiques recommandées pour la programmation JavaScript, développé par l'entreprise Airbnb. Il est largement utilisé par la communauté JavaScript et fournit des directives claires pour écrire un code JavaScript clair, lisible, performant et maintenable.

Question 5

Le guide Airbnb de JavaScript diminue les performances en travail de groupe, mais augmente les performances en travail individuel.

- Vrai
- ✓ Faux

Le guide Airbnb de JavaScript est justement très utile lors du travail de groupe, car il permet que tout le monde code avec les mêmes pratiques, et donc le code sera plus unifié.

Solution n°2

[exercice p. 12]

Question 1

Il est recommandé d'utiliser des variables globales en JavaScript autant que possible.

- Vrai
- ✓ Faux

Il est recommandé d'éviter autant que possible les variables globales en JavaScript, car cela peut entraîner des conflits et des bugs dans le code.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Il est important d'utiliser des commentaires dans le code JavaScript pour expliquer le but et le fonctionnement du code.

✓ Vrai

Faux

Les commentaires aident les autres développeurs (et vous-même à l'avenir) à comprendre le but et le fonctionnement du code.

Question 3

Quelle sont les meilleures façons de déclarer une variable en JavaScript ?

A `let variableName;`

B `var variableName;`

C `const variableName;`

Il est recommandé d'utiliser `const` autant que possible, car cela garantit que la variable ne sera pas réaffectée accidentellement.

Question 4

Les boucles imbriquées sont une bonne pratique.

Vrai

✓ Faux

Les boucles imbriquées pourraient générer des pertes de performance. Il vaut mieux utiliser les méthodes de la classe Array.

Question 5

Il est recommandé d'utiliser des boucles `for-in` pour itérer sur les tableaux en JavaScript.

Vrai

✓ Faux

Tous droits réservés à STUDI - Reproduction interdite

Il est recommandé d'utiliser des boucles `for-of` pour itérer sur les tableaux en JavaScript, car cela garantit que chaque élément du tableau sera traité une fois et une seule. Les boucles `for-in` peuvent entraîner des bugs si des propriétés non numériques sont présentes dans le tableau.

[exercice p. 14] Solution n°3

- Remplacer `var` par `const` pour déclarer les variables immuables (data, resultElement).
- Remplacer la boucle `for-in` par la méthode `forEach()` pour parcourir le tableau data.
- Utiliser la méthode `every()` au lieu d'une boucle pour vérifier si tous les éléments du tableau ont été traités.

Code corrigé :

```
const data = [
    {id: 1, value: "Item 1"},
    {id: 2, value: "Item 2"},
    {id: 3, value: "Item 3"}
];

const resultElement = document.getElementById("result");

function processItems() {
    data.forEach((item) => {
        const listItem = document.createElement("li");
        listItem.innerHTML = `ID : ${item.id}, Value : ${item.value}`;
        resultElement.appendChild(listItem);

        item.processed = true;
    });
}

function checkProcessedItems() {
    const allProcessed = data.every((item) => item.processed);
    return allProcessed ? "All items processed" : "Some items not processed";
}

processItems();
console.log(checkProcessedItems());
```

[exercice p. 15] Solution n°4

Voici le résultat attendu :

```
const products = [
  { name: 'T-shirt', price: 20 },
  { name: 'Jeans', price: 50 },
  { name: 'Sneakers', price: 80 }
];

for (const product of products) {
  console.log(`${product.name} - ${product.price}`);
}
```


Tous droits réservés à STUDI - Reproduction interdite

Solution n°5

[exercice p. 15]

Question 1

L'utilisation de bonnes pratiques en JavaScript peut réduire le temps de développement.

Vrai

✓ Faux

Les bonnes pratiques en JavaScript, comme la modularisation et la gestion des erreurs, demandent un investissement initial, notamment pour mettre en place des outils et adopter les bonnes méthodologies, ce qui peut rallonger le temps de développement.

Question 2

L'utilisation de bonnes pratiques en JavaScript peut améliorer la qualité du code.

✓ Vrai

Faux

Les bonnes pratiques en JavaScript, telles que l'utilisation de noms de variables significatifs et la séparation des responsabilités dans le code, permettent de rendre le code plus facile à comprendre et à maintenir, ce qui améliore la qualité du code.

Question 3

Quel est le but principal d'ESLint ?

Traduire le code JavaScript

Faire de l'autocomplétion dans le code JavaScript

✓ Fournir des suggestions pour améliorer la qualité du code JavaScript

ESLint est un outil de linting qui permet de détecter les erreurs potentielles et de fournir des suggestions pour améliorer la qualité du code JavaScript.

Question 4

Quel est l'avantage de l'utilisation de `let` et `const` plutôt que `var` en JavaScript ?

Elle permet de définir des variables globale

Tous droits réservés à STUDI - Reproduction interdite

Elle permet de définir des variables locales

✓ Elle permet de mieux contrôler la portée des variables

L'utilisation de `let` et `const` en JavaScript permet de mieux contrôler la portée des variables et d'éviter les bugs liés à la réutilisation de variables.

Question 5

Quel est l'avantage de l'utilisation de la méthode `map` pour itérer sur les tableaux en JavaScript ?

Elle permet de modifier les éléments du tableau

Elle permet de filtrer les éléments du tableau

✓ Elle permet de créer un nouveau tableau en appliquant une fonction à chaque élément du tableau d'origine

La méthode `map` en JavaScript permet de créer un nouveau tableau en appliquant une fonction à chaque élément du tableau d'origine, ce qui est utile dans de nombreux cas.

Tous droits réservés à STUDI - Reproduction interdite